

Arquitectura limpia en CSS

Cómo organizar tus hojas de estilo para que sigan siendo fáciles de mantener cuando el proyecto crece. Sin frameworks, solo CSS que ya funciona en todos los navegadores actuales.

1 Ordena la cascada con @layer

Declara el orden de las capas una sola vez, al inicio. Una capa posterior gana a una anterior sin importar la especificidad de los selectores. Los estilos que quedan fuera de toda capa ganan a todas.

utilities	.p-0, .text-center
components	.card, .btn, .nav
layout	grid, contenedores
base	html, body, h1, a
reset	limpia estilos del navegador

Abajo, menos prioridad · Arriba, más prioridad

```
@layer reset, base, layout, components, utilities;

@layer components {
  .btn { padding: var(--space-2) var(--space-4); }
}

@layer utilities {
  .p-0 { padding: 0; } /* gana a .btn */
}
```

2 Tokens de diseño en :root

Define colores, espacios, radios y tipografía como custom properties. Cuando el cliente cambia la marca, editas un solo lugar.

EVITA

```
.header { background: #14213d; }
.footer { background: #14213d; }
.btn    { background: #14213d; }
```

MEJOR

```
:root {
  --color-primary: #14213d;
  --space-4: 1rem;
  --radius: 0.5rem;
}

.header,
.footer,
.btn { background: var(--color-primary); }
```

3 Nombres de clase con BEM

Bloque, elemento y modificador. El nombre dice a qué componente pertenece cada clase, y el selector queda plano: una sola clase, misma especificidad en todo el proyecto.

CLASE	QUÉ ES
.card	Bloque: componente independiente
.card__title	Elemento: parte del bloque
.card--featured	Modificador: variante del bloque

EVITA

```
.card .content h3 { }
```

MEJOR

```
.card__title { }
```

4 Especificidad bajo control

Mientras más baja y pareja sea la especificidad, menos reglas se pisan entre sí. Se lee como (IDs, clases, etiquetas).

SELECTOR	PESO
#menu .link	1,1,0 No uses IDs para estilos
.card .content h3	0,2,1 Depende del HTML
.card__title	0,1,0 El objetivo
:where(.nav) a	0,0,1 :where() pesa 0; ideal en base

Deja !important solo para la capa de utilidades, donde la regla siempre debe ganar.

5 Tamaños fluidos con clamp()

Un valor que escala entre un mínimo y un máximo según el ancho de la pantalla. Reemplaza varios media queries de tipografía y espaciado.

```
:root {
  --text-base:
    clamp(1rem, 0.95rem + 0.25vw, 1.125rem);
  --text-h1:
    clamp(2rem, 1.5rem + 2.5vw, 3.5rem);
}

h1 { font-size: var(--text-h1); }
```

clamp(mínimo, valor preferido, máximo). Usa rem en el mínimo y el máximo para respetar el zoom del usuario.

6 Grid responsivo sin media queries

auto-fit crea tantas columnas como quepan. min() evita que la tarjeta desborde en pantallas angostas. El espacio entre elementos lo pone el contenedor con gap.

```
.grid {
  display: grid;
  grid-template-columns:
    repeat(auto-fit, minmax(min(16rem, 100%), 1fr));
  gap: var(--space-4);
}
```

EVITA

```
.card { margin: 0 16px 16px 0; }
```

MEJOR

```
.grid { gap: var(--space-4); }
```

7 Modo oscuro cambiando solo tokens

Si los componentes usan variables, el modo oscuro consiste en redefinir esas variables. No tocas ningún componente.

```
:root {
  --color-bg: #ffffff;
  --color-text: #1a1a1a;
}

@media (prefers-color-scheme: dark) {
  :root {
    --color-bg: #121212;
    --color-text: #eaeaea;
  }
}

body {
  background: var(--color-bg);
  color: var(--color-text);
}
```

8 Revisión antes de hacer commit

- ☐ ¿Queda algún color o medida escrita a mano que debería ser un token?
- ☐ ¿Algún selector depende de la estructura del HTML, como `.card .content h3`?
- ☐ ¿Usaste un ID o `!important` fuera de la capa de utilidades?
- ☐ ¿Cada regla está en la capa que le corresponde?
- ☐ ¿Los márgenes entre elementos salen de `gap` o de un token de espacio?
- ☐ ¿La página se ve bien entre 320 px y 1440 px de ancho?

9 Un archivo por responsabilidad

Cada archivo tiene un propósito y entra en su capa desde main.css. Los tokens van sin capa: solo declaran variables.

```
css/
├─ main.css
├─ tokens.css
├─ reset.css
├─ base.css
├─ layout.css
├─ components/
│  ├─ button.css
│  └─ card.css
└─ utilities.css

/* main.css */
@layer reset, base, layout, components, utilities;
@import url("tokens.css");
@import url("reset.css") layer(reset);
@import url("base.css") layer(base);
@import url("layout.css") layer(layout);
@import url("components/button.css") layer(components);
@import url("components/card.css") layer(components);
@import url("utilities.css") layer(utilities);
```

@import va antes de cualquier otra regla, salvo @charset y @layer. En producción, une los archivos con Vite.